

Lab 3: Architectural Pattern Selection – Justification

Vaccination Cohort & Dose Scheduling System

Architecture Selection:

We chose **Microservices Architecture** for the **Vaccination Cohort & Dose Scheduling System**.

The system is split into independently deployable services behind an API Gateway: Citizen Registration, Slot Booking, Vaccination Record and Certificate. Layered was rejected because one deployable unit cannot scale verification and booking separately. Client-Server was rejected because a single central server is a bottleneck and single point of failure for a public health platform.

Two Reasons for the Choice :

- **Different load patterns need independent scaling.** Slot booking peaks when a rollout is announced, while QR certificate verification (NFR-001) peaks at venues and events. Microservices let the Certificate Service and Slot Booking Service scale separately instead of scaling the whole application.
- **Fault isolation and independent deployment.** If the Slot Booking Service fails, officers can still record doses (FR-004) and verifiers can still verify certificates. Each business rule, such as the 28-day interval check (FR-001), can be updated and deployed without touching the other services.

Security Advantage :

All client traffic enters through the API Gateway, which authenticates users (JWT) and enforces role-based access before any service is reached, so only authorised users can see citizen vaccination and demographic data (NFR-002). Services are not exposed directly, and each owns its own datastore. The certificate signing key is held only by the Certificate Service, which limits the impact of a breach elsewhere.

Performance Benefit :

Verification of a digitally signed QR code needs only the public key and the signed payload, with no database lookup. The Certificate Service can therefore be scaled horizontally and can verify offline or online in under 150 ms during peak load (NFR-001), without competing with booking or registration traffic.

Trade-off Acknowledged :

Microservices add network latency and data-consistency challenges. We limit this by having each service own its data and exchange only what is needed through defined interfaces, for example Slot Booking reads the Dose 1 date through the Dose History Interface instead of sharing a database.